

P2876R1

Proposal to extend `std::simd` with more constructors and accessors

Published Proposal, 2024-05-22

This version:

<https://isocpp.org/files/papers/P2876R1.html>

Authors:

[Daniel Towner](#) (Intel)

[Matthias Kretz](#) (GSI)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to provide additional constructors and accessors for `std::simd`

Table of Contents

1 Motivation

2 Bitset

2.1 Implementation experience

3 Conversion to and from integral value bit representations

3.1 Building a mask from a compact bit representation

3.2 Compact bit accessor

3.3 Implementation experience

4 Initialising from a list

4.1 Variadic constructor

4.2 Initialisation-list constructor

5 Wording

6 Revision History

References

Informative References

§ 1. Motivation

The [P1928R3] proposal outlines a number of constructors and accessors which can be used to move data in and out of a `simd` object. However, there are a number of other data types in the standard C++ libraries which provide a form of data-parallel value storage, such as `std::bitset` and types which implement the `std::contiguous_range` concept such as `std::span`. It is desirable to be able to easily convert those to and from `std::simd` values too. In this paper we shall examine the benefits of providing constructors for building `simd` or `simd_mask` values from other types, and for adding accessors which allow them to be converted back into other types from their equivalent SIMD values. Some of these proposals were briefly outlined in [P2638R0], and some proposals are new.

§ 2. Bitset

`std::bitset` has many characteristics in common with a `simd_mask` value and it is useful to be able to use a `std::bitset` value as a mask and vice versa. There is no easy way for a programmer to concisely and efficiently achieve these interchange conversions so we propose that a constructor and an accessor are provided in `std::simd`. Firstly, a constructor can be provided for a `simd_mask`:

```
constexpr simd_mask(const std::bitset<size>& b);
```

This constructor allows a bitset with the same number of elements as a SIMD value to be used to build a `simd_mask` for that value. Each element in the constructor mask has the same boolean value as the respective element from the incoming bitset. It is not marked as explicit, so a bitset can be conveniently used anywhere that a `simd_mask` could be.

A `simd_mask` can be converted into an equivalent `std::bitset` using a conversion operator:

```
constexpr simd_mask::operator std::bitset<size>() const;
```

or alternatively through a named method which makes it explicitly clear that the conversion is happening:

```
constexpr std::bitset<size> simd_mask::to_bitset() const;
```

The output bitset value will have the same size as the `simd_mask`, and every element in the bitset will have the same value as its respective `simd_mask` element.

§ 2.1. Implementation experience

When working with ABIs which already have compact bit representations (e.g., AVX512 predicate registers) then converting to and from bitset is efficient. Conversion to and from wide element representations of masks (e.g., SSE or AVX), is more expensive but the `std::simd` library implementation is able to exploit the internal implementation details to make it more efficient than anything that the user could do using the public `std::simd` API.

§ 3. Conversion to and from integral value bit representations

There are several ways that `simd_mask`-like values could be stored, ranging from wide-element values (e.g., SSE, AVX), compact mask (e.g., ARM or AVX-512 predicates), bitsets, or byte-valued memory regions (e.g., `std::simd_mask::copy_to`). The `std::simd` API already has the ability to convert wide elements to and from `simd_mask`s using, for example, the `operator+`. However, it can be useful to be able to convert to and from compact masks represented using raw bits stored in an integral value (something which `std::bitset` also supports explicitly). In this section we propose ways of constructing and accessing packed bits stored in integral values.

§ 3.1. Building a mask from a compact bit representation

When working with SIMD values of fixed sizes (rather than native types whose size can vary by target) it can be useful to express a mask pattern directly. For example, suppose a programmer requires a custom bit pattern, such as `0b10101101011010010101`. There is currently no easy or direct way to encode that pattern into a `simd_mask` value. For example, `simd_mask` has constructors which take a

generator or a byte-valued memory region as input, but neither of these lend themselves to concisely encoding an arbitrary bit pattern.

We propose that the following constructor could be provided:

```
constexpr simd_mask(auto std::unsigned_integral bits) noexcept;
```

The `simd_mask` is constructed such that the first (rightmost, least significant) `M` bit positions are set to the corresponding bit values of `bits`, where `M` is the smaller of `N` and the number of bits in the value representation of `pattern`. If `M` is less than the size of the mask then the remaining bit positions are initialized to zeroes.

The issue with using a `std::unsigned_integral` as the container for the input is that it might contain too many bits for a small mask, or too few bits for a big mask. Without a way of representing an arbitrary number of bits in an integral value (e.g., C23's bit-precise `_BitInt`) we are limited to only allowing up to 64-bits to be inserted into a `simd_mask`, analagous to the same limits as a `std::bitset`.

§ 3.2. Compact bit accessor

There is currently no easy way for mask bits to be extracted from a `simd_mask` in a compact form. Neither of the existing methods to extract is efficient when used for this. For example, a mask's values can be extracted to an array of bytes:

```
auto simd_mask<float> mask = ...;

bool maskAsBytes[1024];
mask.copy_to(maskAsBytes);
```

This copies each mask element to a byte in the supplied output iterator, but there is no easy way to go from that form to a compact bit representation.

The other way to extract the contents of a `simd_mask` is to convert it into a SIMD value representation. In the following example the mask contents are converted to elements which are either 0.0f or 1.0f according to their respective mask bit:

```
auto simd_mask<float> mask = ...;

simd<float> output = +mask;
```

But this doesn't allow easy conversion to a compact set of bits either.

To make it possible to extract compacted bits as an unsigned integral value we propose to borrow an idea from `std::bitset` and provide:

```
constexpr std::uint64_t simd_mask::to_ulong() const noexcept;
```

This will copy up to 64 mask elements to an output value, storing each mask element in a single bit. Unfortunately this potentially loses bits, since `to_ulong` will only emit those bits which can be contained in a 64-bit representation. As with the `unsigned_integral` constructor this could be solved if the C23 bit-precise `_BitInt` type was available in C++.

§ 3.3. Implementation experience

In some problem domains, such as telecommunications, compact representations of bits as integers are very common and it is very important to be able to efficiently convert to and from this format. Providing an API to make it easy to convert masks to and from this representation proved invaluable in writing concise meaningful code in this problem domain.

On a machine with compact mask representation (e.g., we tested on AVX-512) the masks are already stored in compact form, so converting to and from an integer representation was trivial.

On a machine with wide mask representation (e.g., SSE, AVX, AVX2) it is not easy or efficient to use compact representation if only the standard `std::simd` API can be used (e.g., converting to a byte memory and then from that to individual bits). Efficient conversion is only possible if target-specific instructions are used, and the programmer writes non-portable code to use them. For these targets then, it was better that a uniform API into a compact format was provided and handled efficiently by `std::simd` itself.

Constructing and accessing bits through the bitset pathway (e.g., `mask.to_bitset().to_ulong()`) proved also to be inefficient under some conditions. Even on

targets which already had compact mask representation, the extra step in storing data temporarily in a bitset added to the overhead of the conversion. The extra step was difficult to remove because data would be moved in/out of a bitmask in 64-bit blocks, and in/out of a `simd_mask` in blocks whose sizes were governed by the operation in progress. This mismatch in sizes made it difficult to smooth out the data flow across the conversion. Further work will be done to determine if better code generation is possible.

§ 4. Initialising from a list

Inevitably the programmer will want to be able to construct SIMD values with lists of known constants, such as for a lookup table. A reasonable way to achieve this would be to an initialisation list style syntax, as follows:

```
simd<float, 4> myLut = {2.f, 9.f, 23.f, 45.f};
```

In this case the number of values in the initialiser list exactly matches the number of elements in the `simd` value. We recommend that an exact match should be enforced to avoid accidental mismatches. For example:

```
simd<int32_t> table = {1, 2, 3, 4, 5, 6, 7, 8};
```

On an Intel SSE target the default `simd` will have 4 values, so the table is truncated. On an Intel AVX-512 target the `simd` will have 16 values, so some values will be left value initialised (or undefined). Only on an Intel AVX target will the `simd` contain exactly the right number of values. This change in behaviour by a change in target is unlikely to be what the user requires or expects. If the user created a table with a set number of values, then the destination `simd` should match in size. This differs to the initialisation of an array, for example, where setting too few elements will result in other elements being value-initialised.

To permit an initialisation list syntax for `std::simd` it is necessary to create a constructor which allows this style. There are two possible ways to achieve this: with a variadic constructor, and with an explicit initialisation list overload. We look at both of these in turn, and then discuss some of their trade-offs.

§ 4.1. Variadic constructor

A variadic constructor takes a variadic number of arguments and puts each argument into the respective SIMD value element position. Our initial idea for the constructor is to define it as follows:

```
template<typename U0, typename U1, typename... Us>
constexpr simd::simd(U0 u0, U1 u1, Us... us)
requires(sizeof...(Us) == (size - 2));
```

This constructor requires one element to be given for every possible element of the `std::simd` being constructed. Note that two initial arguments are provided, followed by a variadic pack. We believe that having a variadic pack which is easily differentiated to the other common constructor - the broadcast - will reduce compilation times. This constructor is only considered when there are at least two arguments, rather than if it took a single pack which would then need to be considered for every call to a constructor.

§ 4.2. Initialisation-list constructor

An initializer-list constructor would be specified like this:

```
constexpr simd(std::initializer_list<value_type> list);
```

While this does work in principal, the difficulty with it is that it does not allow the size of the source list to be known at compile-time, making it impossible to check that the input source list matches the `simd`'s size. That can only be done by a run-time check, which adds extra overhead.

§ 5. Wording

TBD

§ 6. Revision History

R0 => R1

- Moved range constructor into its own paper [P3299R0]
- Updated discussion of initialiser-list-like constructor.

§ References

§ Informative References

[P1928R3]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 3 February 2023. URL: <https://wg21.link/p1928r3>

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: <https://wg21.link/p2638r0>